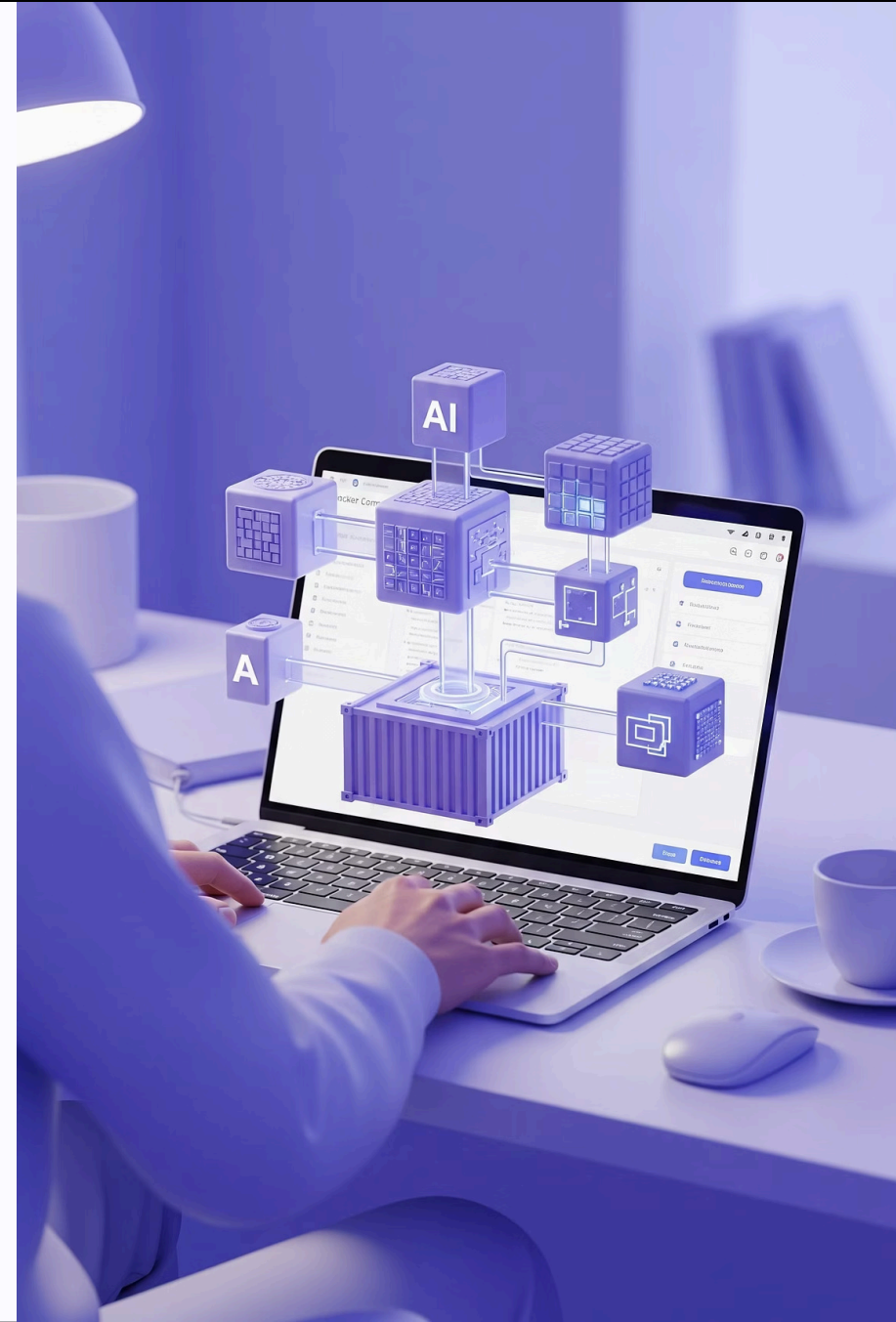


Docker Compose for Multi-Service AI Systems

A practical guide for ML engineers and developers building complex AI applications



Why Docker Compose?

Running multiple containers manually quickly becomes complex and error-prone, especially for AI systems with interdependent components.

One YAML File

Define your entire application stack in a single configuration file

One Command

Spin up your complete development environment with a single terminal command



AI Systems Need Multiple Services

Modern AI applications are rarely single-container deployments. They typically require several coordinated services:

Model Server

PyTorch or TensorFlow Serving for model inference

API Gateway

FastAPI or Flask for handling client requests

Database

Postgres, MongoDB, or Redis for storing data and results

Worker

Background processes for data preprocessing or async tasks

Docker Compose elegantly links these components into a cohesive environment.

Docker Compose Basics

Everything starts with a `docker-compose.yml` file in your project root:

1 Define services, networks, and volumes

Configure each component of your AI system

2 Version control friendly

Keep infrastructure as code alongside your ML code

3 Reproducible ML pipelines

Ensure consistent development environments across team

Common Commands

```
# Start all services  
docker-compose up
```

```
# Run in background  
docker-compose up -d
```

```
# Stop and remove  
docker-compose down
```

Example Compose File

```
version: "3.9"

services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    depends_on:
      - model

  model:
    image: pytorch/pytorch:2.1.0-cuda12.1-cudnn8
    volumes:
      - ./models:/models
```



API Service

Custom build from local directory



Dependency

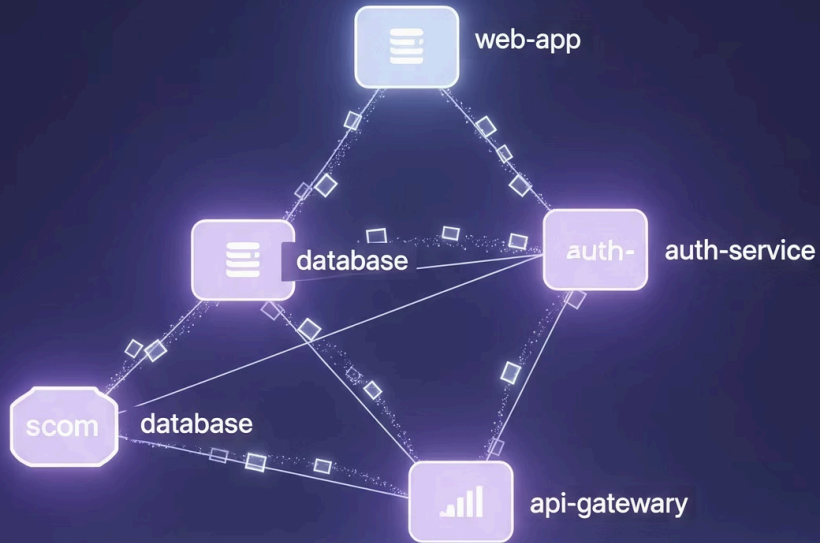
API waits for model to start



Model Service

Pre-built PyTorch image with volume mount

Networking in Compose



Default Network

Compose automatically creates a dedicated network for your application

Service Discovery

Services can call each other using service names:

`http://model:5000/predict`

No Manual IP Management

No need to track container IPs or set up complex networking manually

This simplified networking approach is perfect for distributed AI microservices that need to communicate internally.

Volumes in Compose



volumes:

model-store:

services:

model:

volumes:

- model-store:/models

- **Persistent Storage**

Maintain data across container restarts

- **Shared Resources**

Store model weights, checkpoints, and datasets

- **Training → Inference**

Perfect for AI workflows that generate and then use models



Scaling Services

`docker-compose up --scale api=3`

3x

Replicas

Run multiple instances of the same service



Load Distribution

Spread inference requests across multiple containers



Flexibility

Scale different components independently based on bottlenecks

Benefits for AI Engineers

Docker Compose delivers significant advantages for teams building complex AI systems:



Automation

Eliminates tedious manual container setup and linking



Reproducibility

Ensures ML pipelines work identically across environments



Collaboration

Share complete environments with one YAML file



Local Testing

Test infrastructure locally before cloud deployment



Dev → Prod Bridge

Creates consistency between development and production

Best Practices

- Environment Variables

Use `.env` files for secrets and configuration

- Version Pinning

Specify exact image versions for stability

- Modular YAML Files

Split large systems into multiple compose files

- Health Checks

Configure automatic restarts for failed containers

- Documentation

Document compose workflows in your README

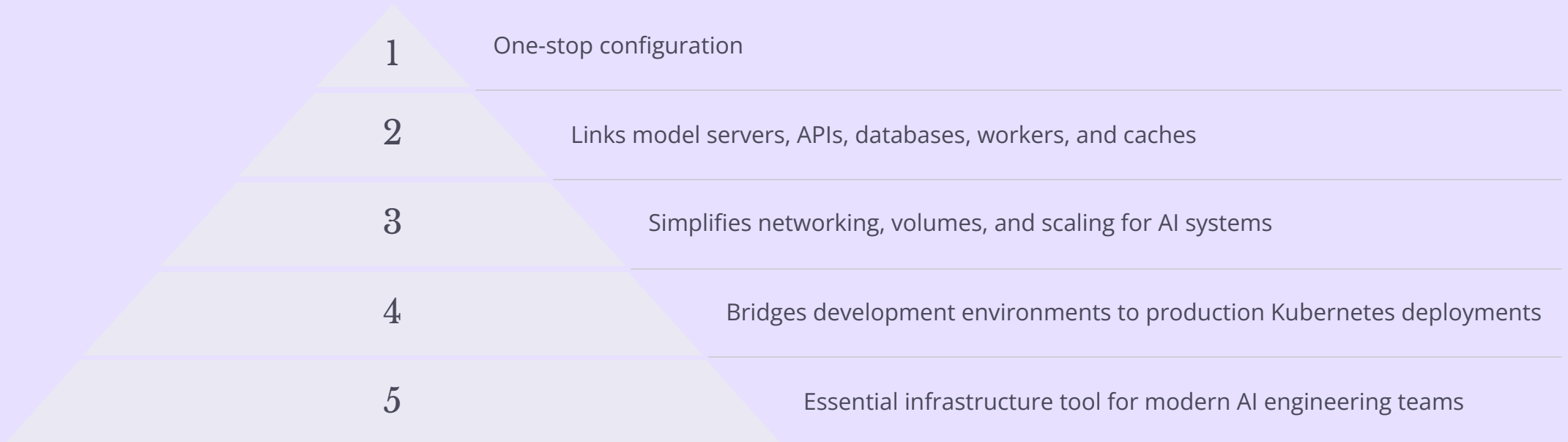
- Resource Limits

Set memory and CPU constraints for stability



Key Takeaways

Docker Compose: The Foundation of Multi-Service AI Applications



Next Steps: Try converting an existing multi-container AI project to use Docker Compose